

USO_IA

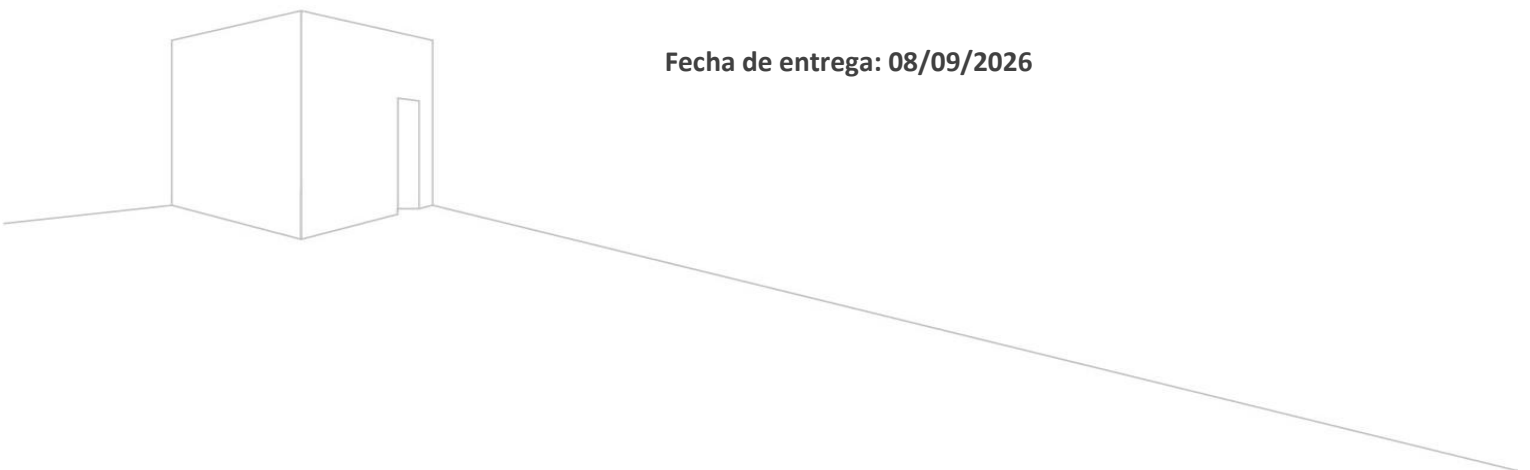
Asignatura: Programación Back end

Sección: TI3041/D-FB50-N4-P13-C1/D

Nombre del docente: Brayan Sebastián Mansilla Pérez

Nombre de los integrantes: Jean La Torre

Fecha de entrega: 08/09/2026



Contenido

I.	Parte 1	3
II.	Parte 2	6

I. Parte 1

1. Configuración de la app y URLs

- **Prompt textual:**

Tengo un proyecto Django con una app llamada catálogo. Indícame cómo registrar catalogo en INSTALLED_APPS dentro de config/settings.py, cómo incluir las URLs de la app en config/urls.py usando include, y genera un catálogo/urls.py con una ruta raíz básica conectada a una vista de prueba

Modifica config/urls.py para incluir la ruta raíz hacia la aplicación catalogo usando include('catalogo.urls')

Escribe el contenido de catalogo/urls.py para mapear la ruta vacía "" a lista_productos y la ruta producto/int:producto_id/ a detalle_producto.

- **Resumen de la respuesta:** Copilot me dio los pasos para registrar catalogo en la configuración global, enlazar las rutas del proyecto y armar el archivo urls.py inicial para manejar el catálogo y los detalles.
- **Usó o modificó antes de integrarla:** Revisé que los nombres de los archivos coincidieran con la estructura de mi proyecto en settings.py y urls.py. Cambié la ruta del include para que cargara desde la raíz y probé que la vista de prueba abriera bien en el navegador.

2. Generación del archivo JSON de productos

- **Prompt textual:**

Genera el contenido de un archivo JSON con 40 productos de ferretería. Debe ser una lista de objetos con las llaves exactas: id, nombre, categoria, precio y stock. Asegúrate de que al menos 8 productos tengan stock igual a 0.

- **Resumen de la respuesta:** Me entregó la lista completa en formato JSON con 40 artículos de ferretería, incluyendo herramientas y materiales con precios, IDs correlativos y 8 productos en stock cero.
- **Usó o modificó antes de integrarla:** Cree el archivo productos.json dentro de catalogo/data/ y pegué el código. Revisé manualmente que vinieran los 40 objetos completos y que las llaves tuvieran los nombres exactos que me pedía la pauta.

3. Imágenes de los productos (Ajuste)

- **Prompt textual:**

Modifica la lista JSON de 40 productos de ferretería para incluir un campo adicional llamado imagen con URLs de imágenes de prueba

Modifica productos.json para que la llave imagen de cada uno de los 40 productos tenga una URL de imagen real y específica según su tipo de herramienta o material, usando Unsplash u otro servicio.

- **Resumen de la respuesta:** Primero me agregó un campo imagen con enlaces genéricos y luego le pedí ajustarlo para que las URLs apuntaran a fotos reales según el tipo de herramienta.

- **Usó o modificó antes de integrarla:** Comprobé que al agregar las URLs no se hubieran desordenado los datos anteriores ni borrado productos. Entré a un par de enlaces para confirmar que cargaban imágenes reales antes de guardar los cambios en el JSON.

4. Vistas para lectura de JSON y búsqueda

- **Prompt textual:**

En `catalogo/views.py`, escribe el código en Python para leer el archivo `catalogo/data/productos.json` usando el módulo `json` sin utilizar base de datos. Crea la vista `lista_productos` que pase todos los productos al template, y la vista `detalle_producto` que busque un producto por su id recibido en la URL

Modifica `catalogo/views.py` para que `lista_productos` reciba un parámetro GET llamado `q` y filtre por coincidencias parciales, sin importar mayúsculas, en el nombre o la categoría..

- **Resumen de la respuesta:** Me generó la lógica en Python usando la librería estándar `json` para abrir y leer el archivo UTF-8, la búsqueda del detalle por ID y el filtro de búsqueda por texto.
- **Usó o modificó antes de integrarla:** No la modifique

5. Plantillas Bootstrap y formulario de búsqueda

- **Prompt textual:**

Dentro de `catalogo/templates/catalogo/`, genera `base.html`, `lista.html` y `detalle.html` usando Bootstrap 5. La base debe tener navbar oscuro y bloque content; la lista debe mostrar los productos en una tabla; el detalle debe mostrar la foto grande y los datos en una tarjeta

Modifica `catalogo/templates/catalogo/lista.html` para agregar un formulario GET con un input de búsqueda y botón "Buscar" usando el término `q`.

- **Resumen de la respuesta:** Diseñó la estructura visual de las páginas en HTML usando Bootstrap 5, aplicando herencia de plantillas (`extends`) y creando la barra de búsqueda en la cabecera del listado.
- **Usó o modificó antes de integrarla:** Creé la carpeta `templates` y guardé los archivos. Tuve que revisar y ajustar las etiquetas `{ url }` de Django dentro del HTML para asegurarme de que los botones de "Ver detalle" y el botón de "Buscar" apuntaran a las rutas correctas.

6. Autenticación, edición y creación de datos

- **Prompt textual:**

Modifica `catalogo/views.py` para agregar `login_view` y `logout_view` usando `django.contrib.auth`. Agrega `editar_producto(request, producto_id)` y `crear_producto(request)` protegidas con `@login_required`, capaces de leer y guardar cambios en `productos.json`.

- **Resumen de la respuesta:** Me entregó las funciones para manejar el inicio y cierre de sesión de usuarios de Django, además de las vistas protegidas para sobrescribir y agregar objetos dentro del JSON.
- **Usó o modificó antes de integrarla:** No la Modifique

7. Formulario prellenado y vistas de Login

- **Prompt textual:**

Genera el código HTML para `login.html` y `formulario_producto.html` usando Bootstrap 5. El login debe tener usuario y contraseña; el formulario de producto debe tener nombre, categoría, precio, stock e imagen

Actualiza `formulario_producto.html` para que contenga inputs etiquetados prellenados con los datos del producto seleccionado si se está editando.

- **Resumen de la respuesta:** Me dio las plantillas HTML para el formulario de login y para la vista de edición/creación con controles de formulario estilizados con Bootstrap.
- **Usó o modificó antes de integrarla:** En el formulario del producto me aseguré de que los atributos name de los campos coincidieran con los que leía la vista en Django.

8. Rutas de autenticación y eliminación de datos

- **Prompt textual:**

Actualiza `catalogo/urls.py` para incluir `login/`, `logout/`, `producto/editar/int:producto_id/` y `producto/crear/`

Agrega una vista `eliminar_producto` en `catalogo/views.py`, protegida con `@login_required`, que reciba `producto_id`, elimine el producto de `productos.json`, guarde la lista actualizada y redirija a `lista_productos`

Incluye en `catalogo/urls.py` la ruta `producto/eliminar/int:producto_id/` y en `lista.html` una columna de acciones visible solo cuando `user.is_authenticated`, con un botón “Eliminar” para cada producto.

- **Resumen de la respuesta:** Completó las rutas que faltaban en el proyecto e implementó la función para remover elementos de la lista JSON y guardar el archivo actualizado, protegiendo los botones en la interfaz.
- **Usó o modificó antes de integrarla:** Probé la vista eliminando un producto de prueba y verificando que desapareciera del JSON sin romper el archivo.

9. Métricas de inventario y destacada visual

- **Prompt textual:**

Modifica lista_productos para calcular total_registros y la cantidad de productos con stock mayor a 0, llamada con_stock, pasando ambas variables al contexto

En lista.html, agrega una tarjeta informativa al inicio que muestre el total de productos y la cantidad con stock disponible. Aplica table-danger a las filas cuyo stock sea igual a 0.

- **Resumen de la respuesta:** Agregó el cálculo del total de elementos y los productos disponibles en la vista, y en la plantilla puso la tarjeta con los datos más la alerta visual en rojo para los sin stock.
- **Usó o modificó antes de integrarla:** No la modifique

II. Parte 2

Al realizar esta evaluación usando Github Copilot con el fin de agilizar nuestro desarrollo de la página web y la generación de datos, principalmente le pedí a la IA que generara el listado de 40 productos en formato Json para no tener que hacerlos manualmente lo cual me genero un gran listado de herramientas con los campos requeridos (nombre, categoría, precio, stock) mientras que en el lado del backend en Django, le solicite las funciones necesarias para leer mi archivo Json con una librería estándar de Python para no usar una base de datos además de hacer las vistas para ver la lista completa.

También le pedi ayuda en varios parámetros como la parte de agregar la barra de búsqueda para filtrar herramientas por nombre o categoria y que implementara las vistas de inicio de sesión con las opciones de crear, editar y eliminar productos, además de resaltar los productos sin stock con el fin de embellecer y mejorar la usabilidad de la página, asimismo me aporoto en la resolución de fallas como en la creación de superusuario ejecutando las migraciones que faltaban.

Gracias a esta evaluación trabajada con la IA pude comprender y manejar mejor Copilot además de entender la dificultad que tiene la mezcla entre front end con back end, incluso me ayudo bastante a solucionar errores cuando me quedaba atascado o cuando necesitaba corregir un Prompt que no había dado el resultado esperado descartándolo o modificándolo.

